

Birmingham City University

LOG REPORT

Data Analytics Pipeline: Planning, Design, and Implementation

Lichess Open Database · MLOps pipeline from plan to implementation

CMP6230 — Formative Draft — Final Individual Report

Student: Nabin Oli

Student ID: 23189629

Programme of study: BSc (Hons) Computer and Data Science

Lecturer: Rupak Koirala

Academic Year 2024 / 25 · 11th June 2026

Abstract

This formative report combines CMP6230 Task 1 (planning and design) and Task 2 (implementation) for a supervised learning pipeline on the Lichess Open Database. Three candidate datasets were evaluated; Lichess was selected for scale, embedded labels, and monthly refresh under a CC0 licence. An initial data storage plan and MLOps pipeline design cover ingestion through monitoring for both notebook analysts and automated training. The final pipeline plan and runnable implementation—orchestrated with Airflow, MinIO, Spark, DuckDB, Feast, Great Expectations, MLflow, FastAPI, and Prometheus—refine that design in the LichessOps repository. The report closes with reflective discussion of plan versus build and further work for the summative submission.

Contents

Abstract	1
1 Candidate source datasets	7
1.1 Dataset one	7
1.2 Dataset two	7
1.3 Dataset three	7
1.4 Chosen dataset and justification	9
1.4.1 Supervised modelling	9
2 Data Storage Plan	10
2.1 Data Source	10
2.1.1 Physical Storage	10
2.1.2 Logical Storage	10
2.2 Ingested Data Storage	11
2.2.1 Logical structure (Star Schema)	11
2.2.2 Justification for the star schema	11
2.3 Alternative structure	13
2.4 Extraction, Loading, and Transformation (ELT)	13
2.4.1 Extract	13
2.4.2 Load	13
2.4.3 Transform	14
2.4.4 Validation	14
3 Initial MLOps and Data Analytics Pipeline Plan	15
3.1 Data ingestion	16
3.2 Data pre-processing	16
3.3 Model development	16
3.4 Model deployment	19
3.5 Model monitoring	19
4 Final MLOps and Data Engineering Pipeline Plan	21
4.1 Shared utilities	21
4.2 Data ingestion	21
4.2.1 What	21

4.2.2	Why	22
4.2.3	How	22
4.3	Data pre-processing	24
4.3.1	What	24
4.3.2	Why	24
4.3.3	How	24
4.4	Model development	26
4.4.1	What	26
4.4.2	Why	26
4.4.3	How	26
4.5	Model deployment	27
4.5.1	What	27
4.5.2	Why	27
4.5.3	How	27
4.6	Model monitoring	28
4.6.1	What	28
4.6.2	Why	28
4.6.3	How	29
5	Data analysis and insight generation / reporting	30
5.1	Introduction	30
5.2	Evidence and reproducibility	30
6	Pipeline Evaluation and Monitoring	32
6.1	Introduction	32
6.2	Evidence and reproducibility	32
7	Reflective Discussion	33
8	Conclusion, recommendation, and future work	34
8.1	Introduction	34
8.2	Evidence and reproducibility	34
9	Further Plans for the Next Submission	36
A	Candidate dataset illustrations	37
B	Lichess Open Database (reference material)	40
B.1	Code index	40
	Bibliography	42

List of Figures

1.1	Candidate dataset comparison across provenance, structure, storage, and supervised-learning affordances.	8
2.1	Star schema for processed Lichess data: a central fact table of games with dimensions for players, openings, and dates.	12
2.2	ELT pattern with object storage.	13
3.1	Pipeline stages: What, Why, Who, and How for each stage of the MLOps workflow.	15
3.2	End-to-end MLOps pipeline divided in terms of configuration, artefacts, and components.	16
3.3	Data ingestion (<i>extract</i> and <i>load</i>): the orchestrator applies ingestion config, downloads the monthly <code>.pgn.zst</code> from Lichess to a raw-data bucket in MinIO, and validates checksums before landing the blob unchanged; checksum failures trigger alerts.	17
3.4	Data preprocessing under orchestration: train/test splits are checked for schema completeness and drift; Great Expectations stores validation reports; failures route to alerts and updated artefact keys, while validated processed data feeds an offline feature store.	18
3.5	Model training: load train/test arrays from validated artefacts, fit the model, compute metrics, register the best run in MLflow, and retain a containerised, reproducible training environment.	18
3.6	Deployment: promote a recent model from MLflow, build a Dockerised FastAPI image, run health checks, and push to the serving host (e.g. EC2); improved metrics can trigger CI/CD to refresh the endpoint.	19
3.7	Monitoring and observability: API metrics scraped by Prometheus, dashboards and automated reports, rule-based routing to Slack/PagerDuty/email, and controlled reruns from user or scheduled inputs.	20
4.1	Data ingestion: download, checksum validation, and load to MinIO raw bucket. .	23
4.2	Pre-processing: validation, feature store, and train/test split under orchestration.	25
4.3	Model training: load splits, fit with hyperparameter search, log to MLflow. . . .	27
4.4	Deployment: promote model from MLflow, serve via FastAPI.	28
4.5	Monitoring: Prometheus, Grafana, drift reports, and pipeline alerts.	29
7.1	Gibb’s reflective cycle for iterative MLOps planning.	33

A.1	Excerpt from a Lichess-rated PGN archive: tournament headers, player ratings, opening metadata, SAN movelist, engine evaluations (<code>%eval</code>), and clock stamps (<code>%clk</code>).	37
A.2	UCI Wine Quality variable inventory: eleven continuous physicochemical features, ordinal <code>quality</code> target (0–10), and <code>color</code> stratum [3].	38
A.3	Boston housing corpus header and fixed-width numeric rows: fourteen tract-level predictors with <code>MEDV</code> as the regression target [4, 5].	39

List of Tables

B.1	Implementation code index with GitHub links.	41
-----	--	----

Chapter 1

Candidate source datasets

Competitive platforms and public benchmarks offer contrasting source-data profiles for supervised learning. This chapter evaluates three candidates transactional game telemetry, tabular chemistry data, and census-derived housing attributes—and selects one for downstream pipeline design. Figure 1.1 summarises three candidates across provenance, structure, storage, and supervised-learning affordances.

1.1 Candidate dataset № 1

Lichess ships monthly rated `.pgn.zst` archives (<https://database.lichess.org/>) with outcomes, clocks, ratings, openings, and SAN movelists under a CC0 licence [1]; see appendix B for inventory. Headers plus ordered moves form the logical model; physically the corpus is compressed text shards, not ready-made tables. Supervised targets include game outcome (`[Result]`) and rating-based signals from pre-game headers (fig. A.1).

1.2 Candidate dataset № 2

Wine Quality [2, 3] is a static CSV of physicochemical assays with ordinal quality scores collected from Portuguese wine samples. Flat relational structure and simple ingestion suit tabular baselines, but the corpus is small, stale, and lacks the temporal refresh needed for operational MLOps practice (fig. A.2).

1.3 Candidate dataset № 3

Boston housing [4, 5] comprises census and environmental attributes for Boston-area tracts with median home value (`MEDV`) as a continuous regression target. The fixed flat-file schema suits classic tabular baselines rather than transactional chess telemetry and offers limited scale for ingestion, validation, and feature-store design (fig. A.3).

Dimension	Lichess open DB (PGN)	Boston house prices (UCI / Kaggle)	Wine quality (Cortez et al.)
What is it?	Millions of rated games with PGN (headers + moves) optional engine evals for some data	Housing feature vectors + house price (USD)	Chemical/physicochemical measures + quality score
Who / What was it collected from?	All are Lichess standard rated games. Dump of the Lichess Game Database	1978 Boston census tract housing; fixed N rows	Portuguese red wine physico-chemical lab samples; fixed N rows
Why was it collected?	Lichess is an open source platform. They publish data for supporting open research + transparency	Classic regression benchmark; policy/real-estate context	Process monitoring / quality control in oenology
Where was it obtained?	https://database.lichess.org (CC0) monthly zipf file in .pgn.zst format	Kaggle and UCI mirrors (open)	Kaggle and UCI (open)
Logical structure	Semi-structured PGN format with headers + movetext	Single flat table (14 predictors, 1 target)	Single flat table (11 features, 1 integer target)
How is it currently stored?	Zstandard-compressed text files. It is object-host downloadable.	CSV/parquet in zip / tabular	CSV in zip / tabular
How could it be used for Machine Learning?	Target: result (3-class) or white_rating_diff; X: Elo, time, ECO, form...	Target: MEDV (reg); X: CRIM, RM, ... (reg)	Target: quality 3–8 (ord./reg); X: pH, alcohol, ...
Ground truth	Posted game result + Lichess rating diff after game (platform)	Harrison–Rubinfeld 1978 survey aggregates	Expert sensory median score
Prior ML use	Widely: outcome prediction, opening analysis, recommendation	Omnipresent UCI baselines	Common classification / regression UCI work

Figure 1.1: Candidate dataset comparison across provenance, structure, storage, and supervised-learning affordances.

1.4 Chosen dataset and justification

We select the Lichess open database for MLOps development due to its sheer volume, scale, embedded labels, CC0 licensing, and monthly refresh. Lichess dataset clearly outweighs the simpler study purpose datasets like Wine Quality and Boston housing baselines (sections 1.2 and 1.3 and fig. 1.1). Additionally, Lichess Data is produced from a live platform with a large user base, so it has a real-world provenance and operational profile that suits and provides ample direction for learning opportunity in MLOps pipeline design and Data Management.

1.4.1 Supervised modelling

The primary supervised task is *pre-game outcome prediction*: given features available before the first move (ratings, time control, variant), estimate whether White wins, Black wins, or the game is drawn, a three-class classification problem with ground truth in `[Result]` (1-0, 0-1, 1/2-1/2).

Chapter 2

Data Storage Plan

This chapter focuses on the data decisions taken while designing the MLOps pipeline. Further this section also discusses about the physical and logical structure of the dataset showcasing how the current design decisions helps in making the optimal for data analytics and downstream meaching learning tasks too.

2.1 Data Source

The data is from Lichess platform which is an open source alternative version of chess.com for playing chess online. Lichess publishes its entire games from database for public use at `database.lichess.org`. It is a bulk file which can be downloaded over HTTPS. Each month a single compressed file is published alongside a SHA256 checksum for validation. This project downloads those files, verifies it, and lands it in the object store named MinIO.

2.1.1 Physical Storage

The raw file downloaded from Lichess which is downloaded and stored in object storage is a compressed PGN file using Zstandard. Zstandard (zstd) was chosen by Lichess because of its balance for compression ration and decompression speed. Once decompressed, the plain-text PGN is roughly five times larger. A typical month goes from 20 GB to 100 GB. The raw file remains in the lake as a single immutable object with no transformation is done at this stage.

2.1.2 Logical Storage

The PGN format is semi-structured file format for storage chess games with metadata. There is no enforeced schema but the structure repeats reliably. Every game is a block of following components:

1. A set of headers
2. An empty line in between
3. The move list in Standard Algebraic Notation. Some games also contains evaluation from stockfish chess engine.

Logically we can think of the dataset as two nested entities.

1. **Game:** Fixed attributes (date, players, result, opening, etc.).
2. **Move:** An ordered list of half-moves, each carrying a move number, SAN notation, UCI coordinates, evaluation, and clock remaining.

The cardinality is one game to zero-or-many moves.

2.2 Ingested Data Storage

After extraction and loading, the raw `.zst` object is transformed into a set of Parquet tables arranged in a star schema. I follow an ELT pattern: the compressed file lands unchanged in MinIO first (load), then Apache Spark reads from object storage to parse and shape the data (transform), and the curated tables are written to MinIO as Parquet and loaded into DuckDB for analytics. Parquet file format has been chosen due to its fast aggregations for columnar database. It also provides features like high compression and predicate push-down with Spark/DuckDB. MinIO has been chosen for object storage due to its S3 compatible format which provides the flexibility to move to production grade object stores like Amazon S3.

2.2.1 Logical structure (Star Schema)

At this stage of data ingestion, processed data is modelled as a star schema centred on `games`. The design is shown in fig. 2.1 as a crow-foot ERD. Justification for the star schema is given in depth in section 2.2.2

2.2.2 Justification for the star schema

The star schema was chosen because it directly supports the two main consumer groups in the pipeline:

Need	How the star schema meets it
ML feature engineering (Feast)	Feast works with entity keys; the fact table and dimensions can be mapped to Feast FeatureViews with simple joins on player, opening, or date keys. Point-in-time joins are straightforward.
Analyst / Data Scientist notebooks	A single join from the fact table to a dimension is enough for most queries; no complex multi-table logic.
Storage efficiency	Player attributes (username, title) are stored once, not repeated per game. Parquet columnar compression reduces the footprint further.
Incremental monthly loads	Partitioning on year/month means new months are appended; dimensions are updated with a merge without rewriting the whole fact table.
Data quality checks	Each table can have a dedicated Great Expectations suite (e.g., null checks on fact columns, uniqueness on dimension keys).

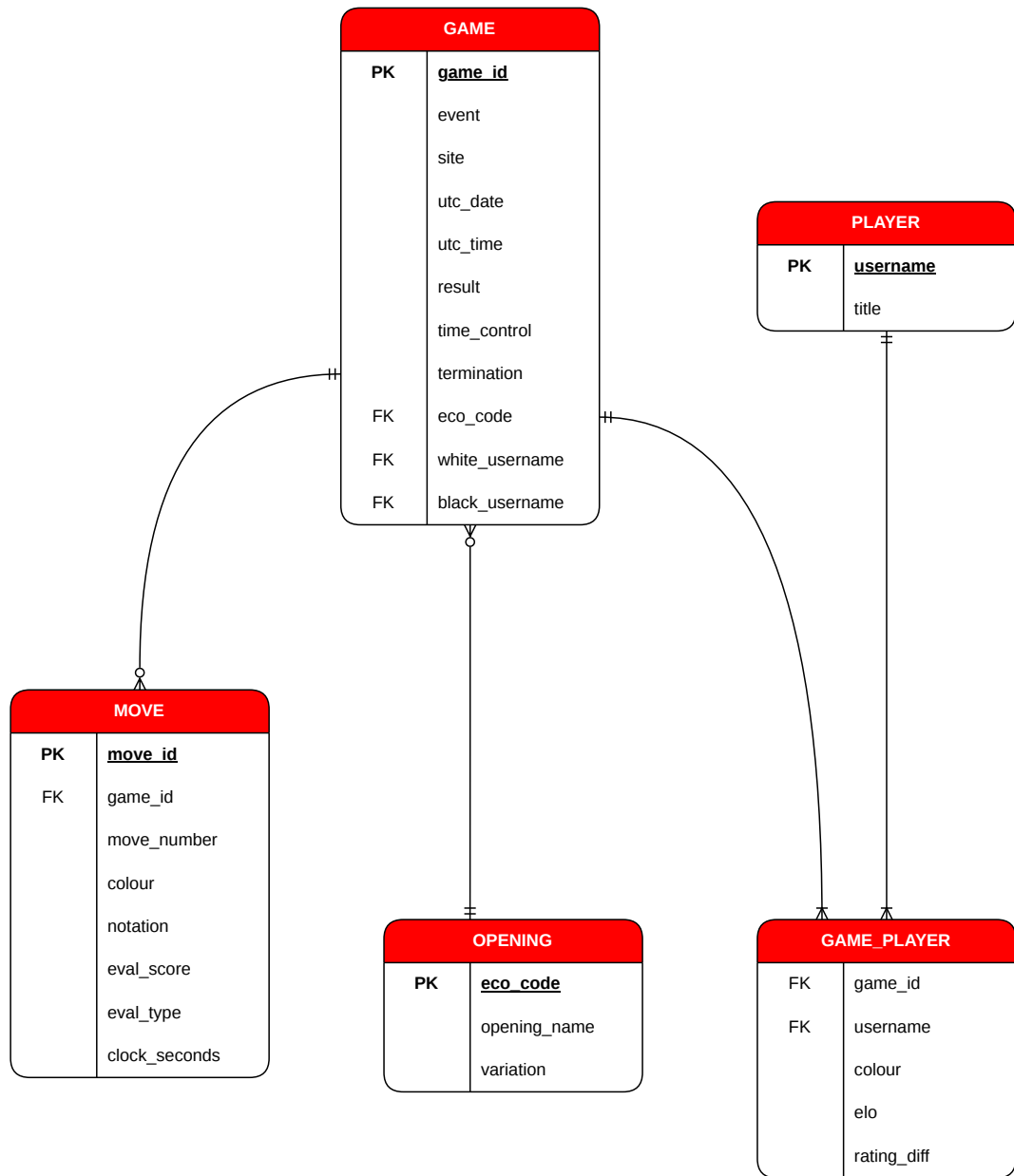


Figure 2.1: Star schema for processed Lichess data: a central fact table of games with dimensions for players, openings, and dates.

2.3 Alternative structure

Approach	Upside	Downside & Mitigation	Why not chosen
One Big Table	Zero joins, simple CSV	Huge duplication, brittle schema, breaks temporal correctness	Duplication and temporal fidelity risk too high
Star Schema (chosen)	Familiar, fits Feast, good compression, incremental	Joins needed; mitigated by columnar scans and small dimensions	—
Snowflake	Even less redundancy in dimensions	Extra joins for minimal gain (shallow ECO hierarchy)	Complexity doesn't pay off
3NF relational	Clean updates, no anomalies	Many-table joins hurt read speed at scale	Append-heavy workload; read performance more important
Medallion layers	Clear quality gates	Overlaps with raw/processed bucket split	Redundant; our raw bucket already provides the “bronze” layer

2.4 Extraction, Loading, and Transformation (ELT)

I follow an ELT pattern because the heavy transformation happens after the data lands in the object store MinIO.

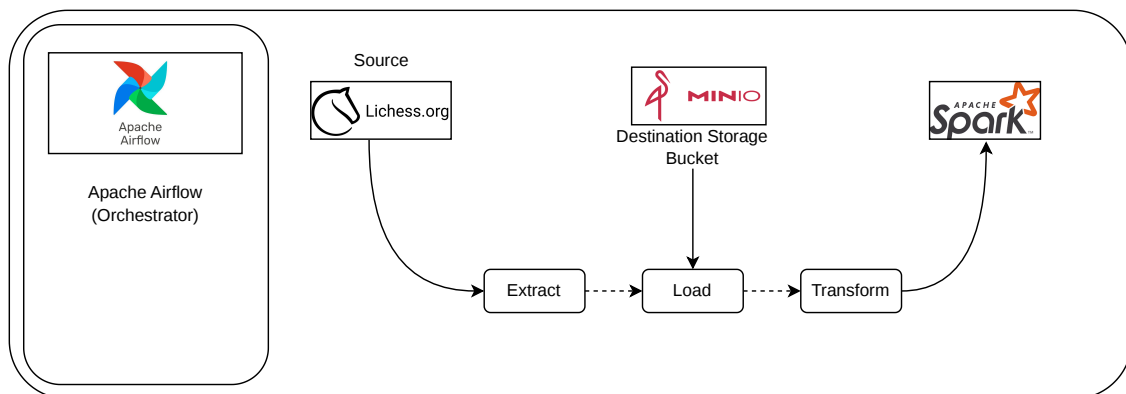


Figure 2.2: ELT pattern with object storage.

2.4.1 Extract

An Airflow DAG fetches each monthly `.pgn.zst` file from Lichess over HTTPS and checks it against the published SHA256 checksum. A mismatch fails the task and triggers an alert.

2.4.2 Load

The verified file lands in MinIO at `s3://lichess-raw/pgn/...` without decompression or parsing. It stays there as an immutable archive until Spark reads it.

2.4.3 Transform

PySpark streams the `.zst` object from MinIO, decompresses it in-flight, and parses games with a python-chess UDF. Headers and derived fields (e.g. move count) go to `fact_games`; half-moves, engine evaluations, and clock times optionally go to `fact_moves`. Distinct players, openings, and dates are merged into `dim_player`, `dim_opening`, and `dim_date`. Curated tables are written to `s3://lichess-processed/` as Parquet, with fact tables partitioned by year and month.

2.4.4 Validation

Great Expectations runs a checkpoint on each new partition (schema, null rates, value bounds). Partitions that pass are released to Feast and analyst notebooks.

Chapter 3

Initial MLOps and Data Analytics Pipeline Plan

This section presents the planned data analytics pipeline for ingesting, processing, and serving the Lichess dataset to two classes of consumer: (1) Analysts performing ad hoc queries and generating reports via SQL/notebooks and (2) An automated machine learning pipeline for supervised model training and serving.

Pipeline stages: What, Why, Who, and How (task sheet Exercise 1 Step 2)				
Stage	What (inputs/output)	Why	Who	How (internal steps/tools)
Data ingestion	Input: new .pgn.zst Output: bronze objects + job metadata	Source of truth late binding to schema catch drift early	Data Engineer	1. Apache Airflow 2. Checksum validation 3. zstd decompression 4. Spark parser 5. Land to Minio bucket
Data pre-processing	Input: raw pgn Output: Delta tables, QA report	ML & SQL need clean types testable contracts	Data Engineer, ML Engineer	1. PySpark 2. Great Expectations 3. MinIO 4. Feast
Model development	Input: versioned feature slice Output: model + metrics in MLflow	Reproducible training avoid skew-serve mismatch	ML Engineer, Data Scientist	1. Temporal train/val/test split 2. LightGBM/XGBoost 3. MLflow (tracking + registry) 4. SHAP explainability 5. Registry gate (metric threshold)
Model deployment	Input: registered model Output: prediction service / batch	Deliver business value controlled rollout	ML Engineer MLOps	1. FastAPI 2. Blue-green deployment 3. shadow traffic / canary 4. Automatic rollback
Model monitoring	Input: live/batch scores + labels Output: alerts, retrain jobs	Drift and Data debt are ML-specific fallout models	SRE, ML engineer, data scientist on-call	1. Evidently (drift, data quality) 2. PSI / KS for drift detection 3. Grafana, Prometheus 4. Airflow retrain DAG

Figure 3.1: Pipeline stages: What, Why, Who, and How for each stage of the MLOps workflow.

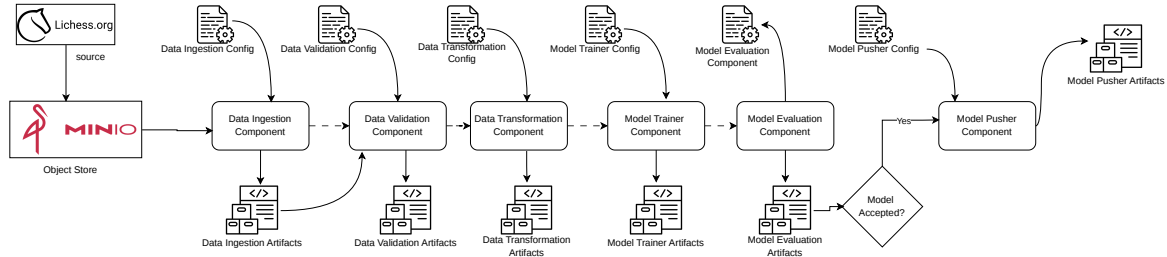


Figure 3.2: End-to-end MLOps pipeline divided in terms of configuration, artefacts, and components.

3.1 Data ingestion

This pipeline automates the ingestion of Lichess’s monthly public game data using the Apache Airflow orchestrator. This stage covers the *extract* and *load* steps of the ELT pattern (fig. 2.2; chapter 2): the monthly shard is downloaded over HTTPS, its integrity is verified (SHA256), and the compressed file is landed in MinIO as a single immutable object with no parsing or reshaping.

Decompression, PGN parsing, and curated star-schema Parquet are performed in preprocessing (section 3.2). We use MinIO as an S3-compatible store for raw and processed data; Parquet partitions in the processed bucket feed DuckDB and downstream analysts and ML tooling.

3.2 Data pre-processing

In preprocessing, a PySpark job implements the *transform* step: it reads the raw `.zst` stream from MinIO, decompresses the Zstandard payload, parses each game into header-derived fields (and optionally move rows), builds the star-schema DataFrames (`fact_games`, dimensions, optionally `fact_moves`), and writes partitioned Parquet to the processed bucket.

After Spark writes those star-schema Parquet files to MinIO, they are loaded into DuckDB, the columnar offline store. DuckDB holds the `fact_games` and dimension tables, enabling fast SQL for both validation and feature generation. A Great Expectations checkpoint runs on the entire DuckDB table (or the underlying Parquet) before any split. It checks column presence, data types, null ratios, and allowed value ranges. If validation fails, the pipeline halts and sends an alert; if it passes, an artefact is produced certifying data quality. Feast (with DuckDB as its offline store) executes point-in-time feature retrieval on the full historical dataset. Time-dependent features (e.g., player win-rate over the last 30 days) are computed using only information available before each game’s timestamp, preventing temporal leakage. The resulting feature DataFrame now containing all features and the target is split chronologically (e.g., 80 % train, 20 % test). Because features were built on the whole dataset before splitting, both partitions contain every column; the “columns missing in training/test” problem is avoided entirely.

3.3 Model development

Training reads transform bundles and logs git, configs, and metrics in MLflow (fig. 3.5). Research approves hypotheses; engineers run sweeps with sequestered test data until final evaluation exports.

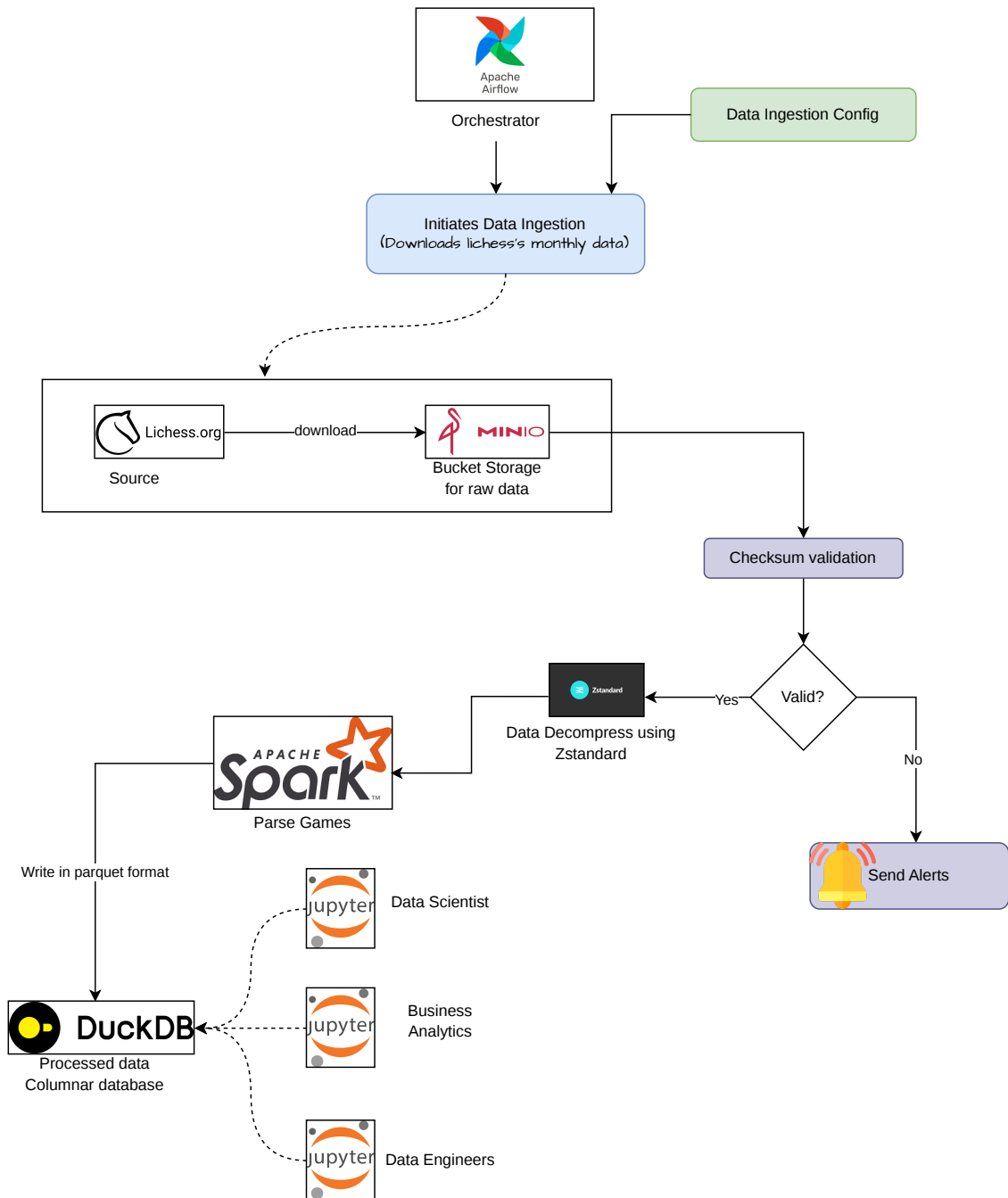


Figure 3.3: Data ingestion (*extract* and *load*): the orchestrator applies ingestion config, downloads the monthly .pgn.zst from Lichess to a raw-data bucket in MinIO, and validates checksums before landing the blob unchanged; checksum failures trigger alerts.

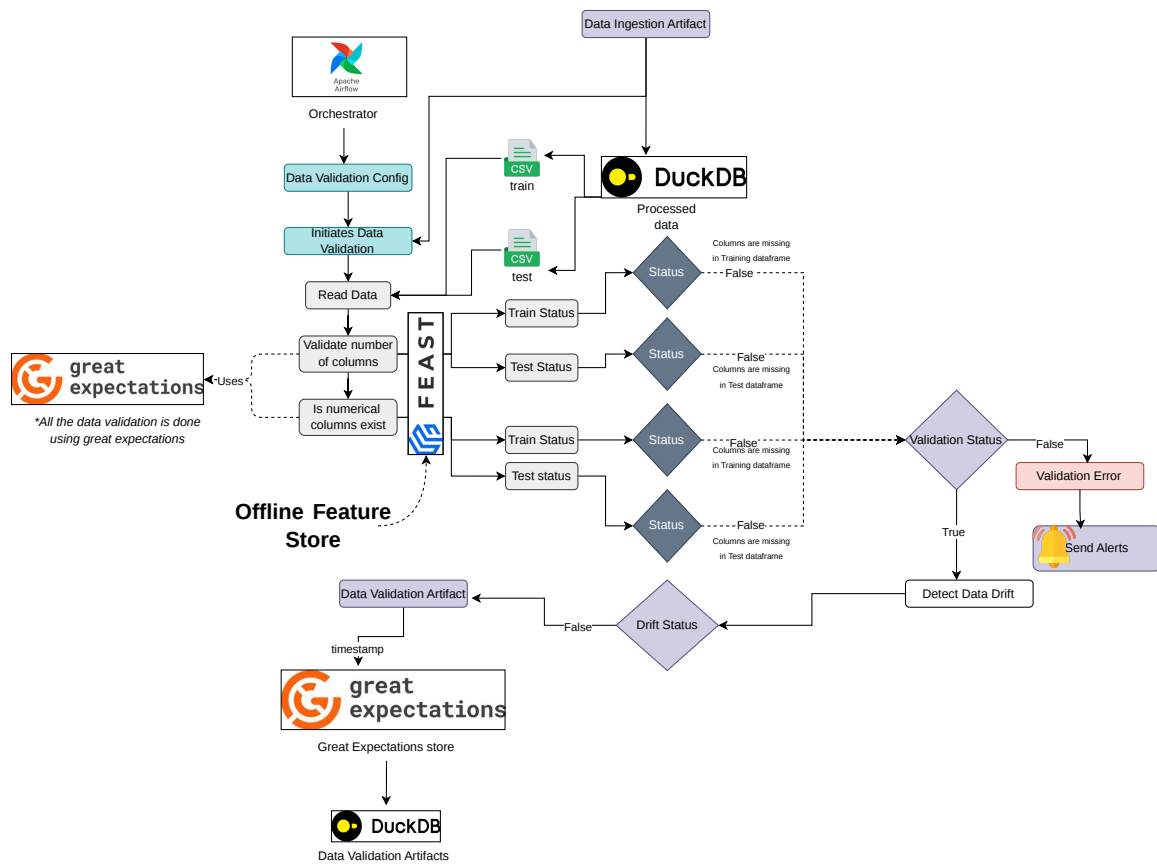


Figure 3.4: Data preprocessing under orchestration: train/test splits are checked for schema completeness and drift; Great Expectations stores validation reports; failures route to alerts and updated artefact keys, while validated processed data feeds an offline feature store.

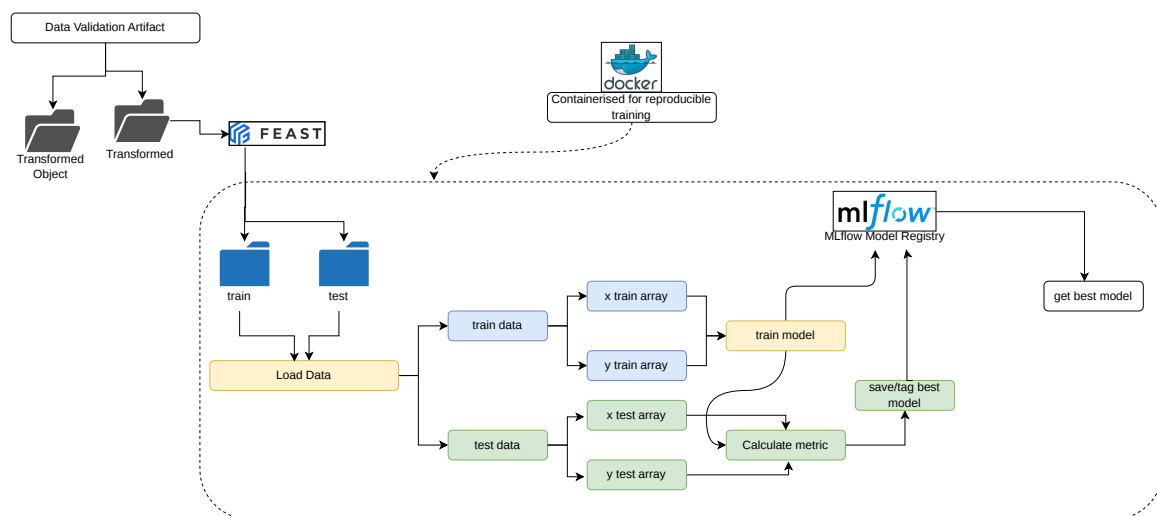


Figure 3.5: Model training: load train/test arrays from validated artefacts, fit the model, compute metrics, register the best run in MLflow, and retain a containerised, reproducible training environment.

3.4 Model deployment

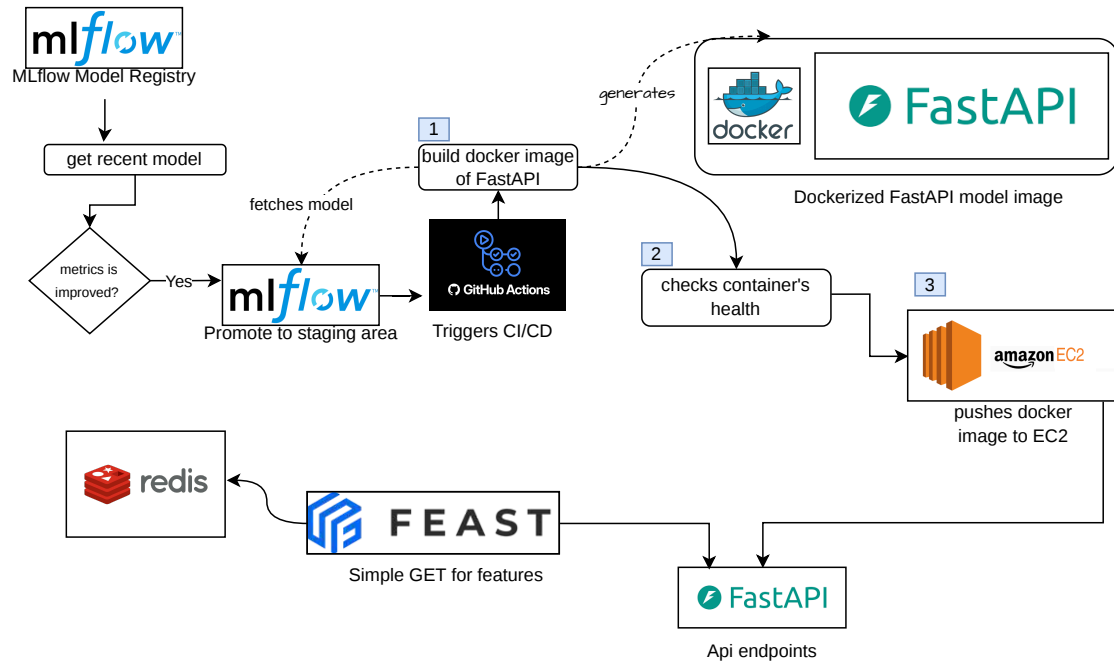


Figure 3.6: Deployment: promote a recent model from MLflow, build a Dockerised FastAPI image, run health checks, and push to the serving host (e.g. EC2); improved metrics can trigger CI/CD to refresh the endpoint.

Containers promote only after gates; MLOps owns CI/CD with SecOps image review (fig. 3.6). Canary and rollback retag registries; contracts keep serving aligned with preprocessing.

3.5 Model monitoring

Production tracks feature drift, latency SLOs, and score error as new months arrive (fig. 3.7). MLOps triages alerts; retrains stay lineage-linked to ingestion batches.

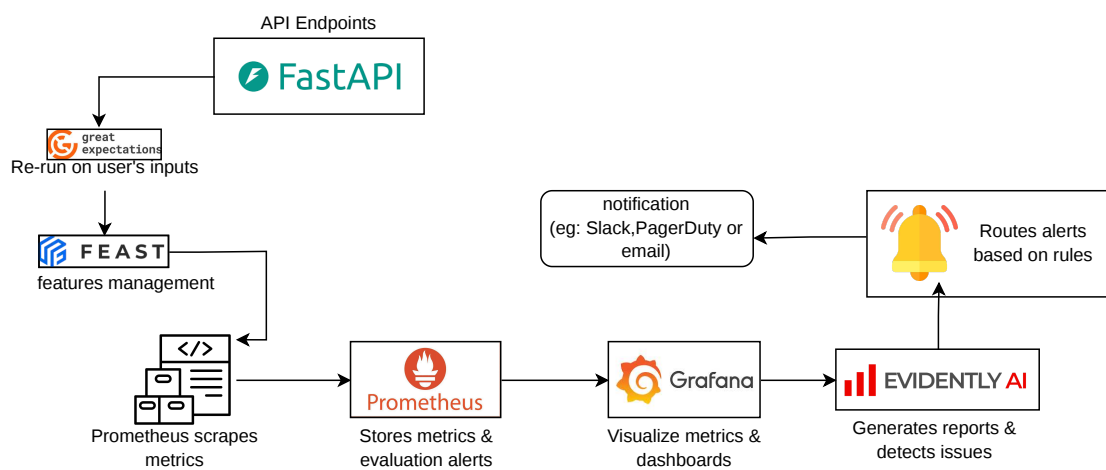


Figure 3.7: Monitoring and observability: API metrics scraped by Prometheus, dashboards and automated reports, rule-based routing to Slack/PagerDuty/email, and controlled reruns from user or scheduled inputs.

Chapter 4

Final MLOps and Data Engineering Pipeline Plan

The following sections document the refined pipeline plan as implemented in the LichessOps repository 🐙[6]. Each stage follows the What, Why, and How structure from the module brief, aligned with the Task 1 diagrams but updated where implementation constraints changed the approach.

4.1 Shared utilities

Before walking through ingestion or training, it is worth describing the small shared library I extracted once several packages started duplicating the same path logic. Early prototype version had several hard-coded directories after repetitive copy-paste, I centralised the behaviour under `libs/shared` making the code reusable across different components.

Each component has configuration loading. The main use of configuration loading is to (🐙[7]) walk up from the package until it finds `pyproject.toml`, then deep-merge `config/default.yaml` which a config file with a package-specific file. As a developer, This pattern eased up profoundly in configuration management and let me modify MinIO endpoints or DuckDB paths in one place during Docker bring-up without touching Python imports. None of these modules are glamorous, but they made the pipeline development more efficient and less error-prone.

4.2 Data ingestion

4.2.1 What

Data ingestion, in this project mainly involves pulling the monthly Lichess database dump from the public facing website <https://database.lichess.org> and storing it in durable object storage so every later stage uses the same bytes. The input is the template URL Lichess publishes from `database.lichess.org` where we download a Zstandard-compressed PGN file [8] named `lichess-YYYY-MM.pgn.zst`. It is a compressed `.pgn` file which is a standard format to represent the chess game. Each game inside this `.pgn` file is text with headers (player names, ratings, opening, result). The output of ingestion is not yet a dataframe. it is that validated archive

sitting in the `lichess-raw` bucket [9] in a format which is ready for Spark for transformation.

The visible components are a Python downloader CLI, checksum validation, boto3 uploads, MinIO provisioned via Docker Compose [10], and Apache Airflow [11] as the scheduler here.

4.2.2 Why

I chose MinIO because PGN and `.zst` blobs do not map cleanly onto a columnar warehouse table. Object storage treats them as opaque files with cheap immutability and no semantics. I can reprocess April 2024 months later without re-downloading if the raw key still exists. Here, Checksum validation matters due to size of raw file. If the checksum fails then early detection prevents issues. Catching these issues early helps to save time and resources early in the pipeline.

Airflow runs the DAG on `0 3 1 * *` (03:00 on the first day of each month) so developer does not need to manually click download when Lichess typically publishes. ELT fits my development setup. Here transformation stays on Spark and DuckDB.

4.2.3 How

The `lichess_monthly_ingestion` DAG [12] shells out to `lichess-data`. That Keeps tasks testable from the terminal with the same flags Airflow passes.

On the default path (`use_elt=True`):

1. **download_shard**: `lichess-data download` calls `download_month` in [13]. The client resumes partial files and can verify SHA256 against Lichess’s published sums while bytes still stream.
2. **upload_raw**: `upload_raw_shard` in [14] pushes to `s3://lichess-raw/` using [15]. When `LICHESS_STORAGE_BACKEND=local`, upload is a no-op for laptop-only runs.
3. **validate_shard**: `validate_checksum` in [16] re-checks integrity. I treat a failure as a hard stop because the DAG should not proceed to Spark on a bad object.
4. Downstream tasks: (`spark_transform`, `duckdb_sync`) consume the raw key which they are preprocessing but depend on this stage finishing cleanly.

Bringing up storage uses the Compose `core` profile from the repo root `docker compose -profile core up -d` which also starts MinIO and a one-shot job that creates `lichess-raw`, `lichess-processed`, and `mlflow-artifacts`. Retrieval for later stages is literally an S3 GET on the month prefix which analysts or Spark both read the same URI via [17].

For retrieval, `raw_s3_uri(month)` returns the full `s3://` key, which DuckDB’s `httpfs` extension or Spark’s S3 configuration consumes. That indirection is how ingestion meets the brief’s “storage and retrieval” wording even before any ML features exist.

When Lichess rotates CDN behaviour, the downloader reads the monthly index rather than hard-coding URLs. That small robustness mattered because archive paths changed between academic years.

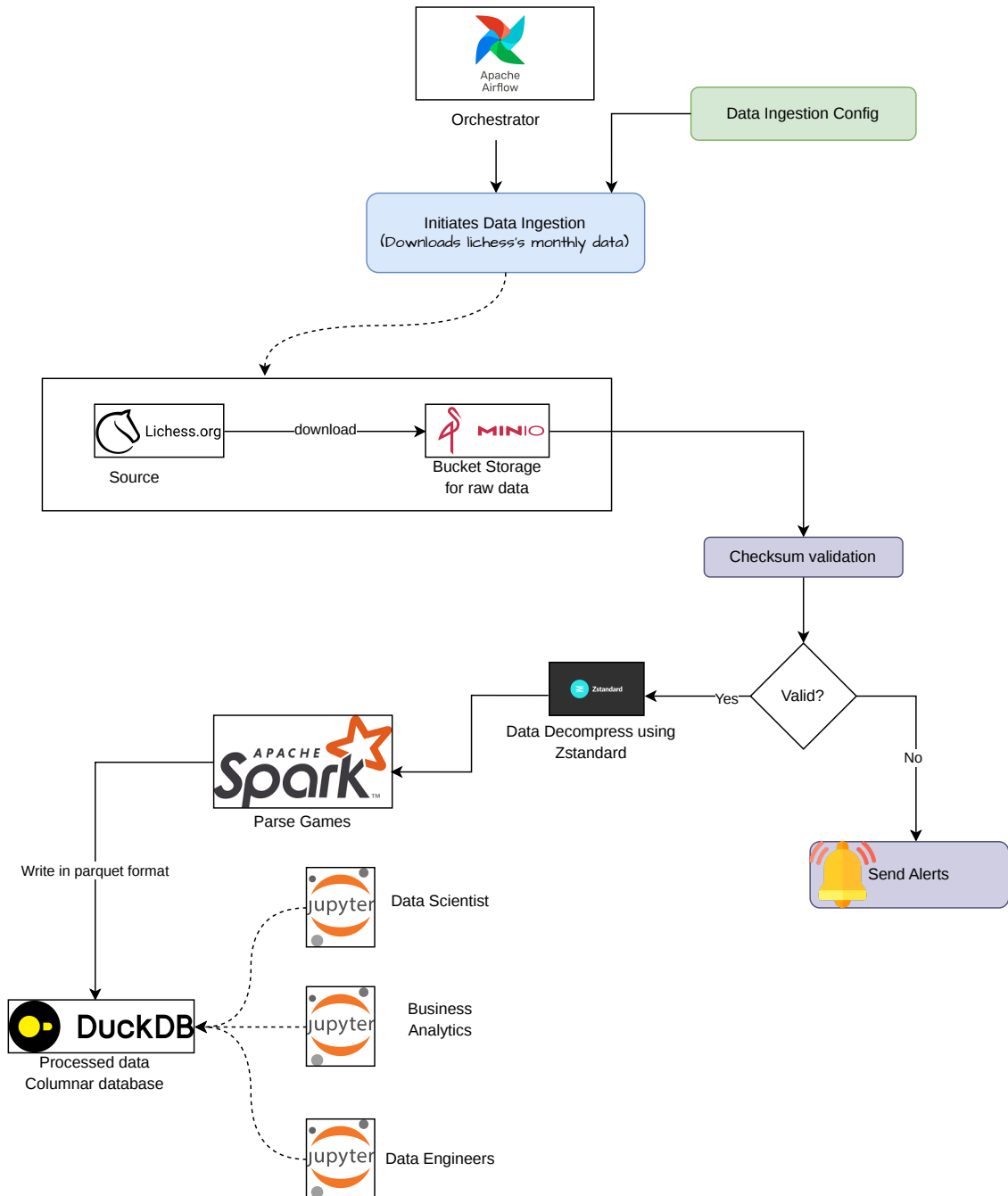


Figure 4.1: Data ingestion: download, checksum validation, and load to MinIO raw bucket.

4.3 Data pre-processing

4.3.1 What

Pre-processing turns the chess games into tabular features which later ML libraries can ingest. Inputs are the raw `.pgn.zst` in MinIO. Outputs include partitioned Parquet under `lichess-processed`, a DuckDB file with star-schema tables which are Pandas-derived feature in Parquet format. Feast train/test datasets, and Great Expectations validation artefacts. If this stage is wrong, the model stage inherits silent bugs. Hence the emphasis has been given on validation before training.

4.3.2 Why

PGN is the lingua franca of chess software but a poor format for data analytics efficiency. It consists of variable header keys, multi-line movetext and comments which doesn't fit the tabular format required for efficient data analysis. We store the final artifact file as Apache Parquet [18] which provides us columnar compression and a schema which is compatible across Spark, DuckDB, and Pandas. Here, DuckDB lets run SQL checks such as (*e.g.* "how many games lack an ECO code?"). For feature storage, I employed Feast. Feast [19] handles point-in-time joins so I do not leak future results into past rows, a mistake I almost made when I was first time computing rolling win rates. Great Expectations [20] encodes data validation expectations as a checkpoints. When a new monthly data arrives with a schema drift, then the pipeline halts the process instead of training on that drifted data.

4.3.3 How

We do following operations in the pre-processing stage:

4.3.3.1 Spark transform

Task `spark_transform` submits work in  [21]. The job streams the Zstandard payload, parses headers into structured fields, builds `fact_games` plus dimension tables, and writes a wide games table to the processed bucket. I use the `pipeline` Compose profile (`core + pipeline`) when RAM allows; otherwise `run_local_transform` keeps development unblocked.

4.3.3.2 DuckDB sync.

 [22] registers Parquet from MinIO through `httpfs` and exports a single wide table for the Python feature code. Analysts can attach the same `.duckdb` file in the CLI container (`tools` profile) and run ad hoc SQL—that was the “interactive consumer” from Task 1.

4.3.3.3 Pandas pipeline.

`preprocess_features` executes ten stages defined in  [23] via `PIPELINE_STAGES`: (1) `parse_event` maps Lichess event strings to Bullet/Blitz/Classical; (2) `encode_result` turns 1-0, 0-1, 1/2-1/2 into class labels; (3)–(4) date and clock features; (5) `impute_ratings` for missing values; (6) title flags; (7) opening and ECO family; (8) `add_historical_pregame_features` for rolling player

form; (9) move-count summaries. I kept each step as a function so unit tests could target one transformation at a time.

4.3.3.4 Legacy extract.

With `use_elt=False`,  [24] and a streaming Parquet writer avoid Spark entirely. It is slower on full months but invaluable on a train journey with only Docker Desktop and 8 GB RAM.

4.3.3.5 Feast split.

 [25] materialises pre-game features with timestamps respected, then applies an 80/20 chronological cut—no random shuffle, because random splits inflate metrics on temporal game data. Both partitions retain the same columns; I learned that lesson after an earlier split dropped sparse opening columns from the train set only.

4.3.3.6 Great Expectations.

`validate_ge` invokes  [26]. Failed expectations block `train_model`. The GE Postgres container (`ge` profile) stores suite metadata; the actual run still happens via host CLI, which is slightly awkward but workable for coursework scale.

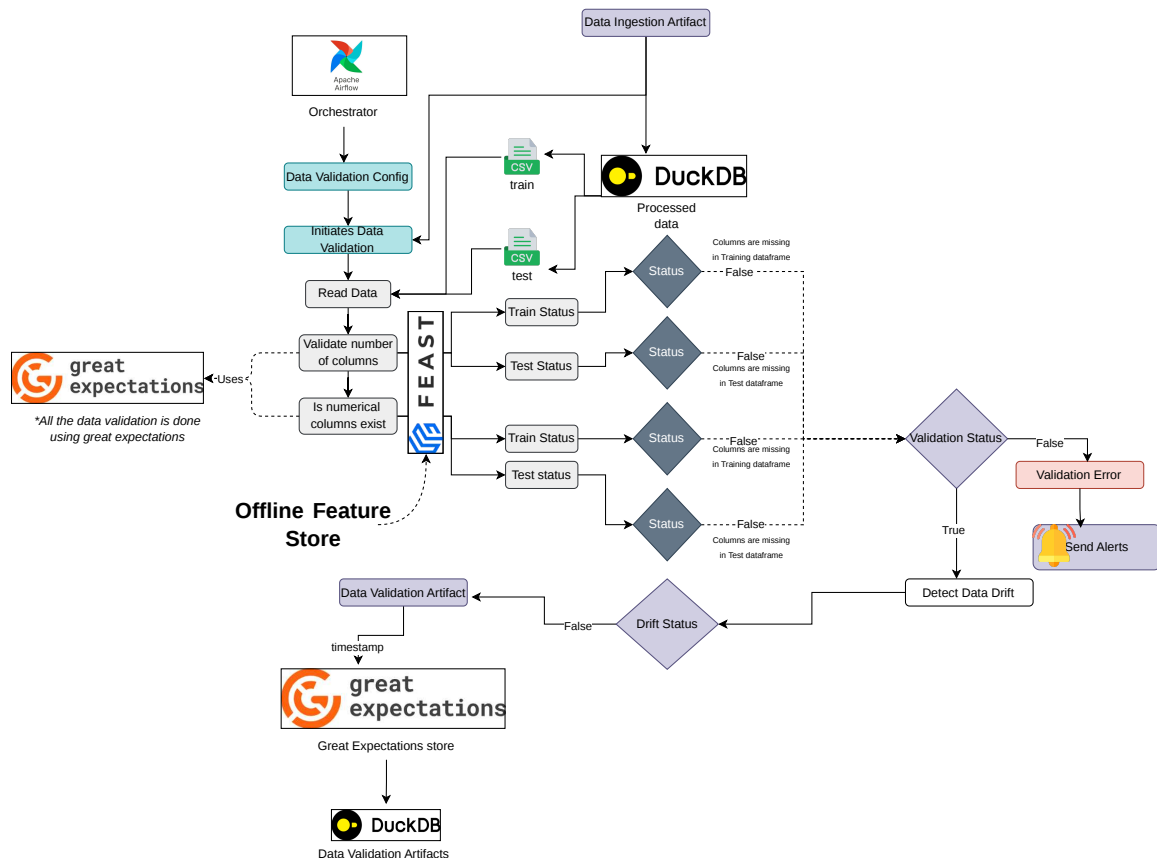


Figure 4.2: Pre-processing: validation, feature store, and train/test split under orchestration.

CSV was tempting for quick plots but punishes wide schemas. In our case, Parquet’s column

pruning matters when move-level features widen the table. The pre-processing stage is where I spent the most calendar time, mostly on edge cases like unrated players, abandoned games, transpositions with missing ECO which do not show up in textbook datasets.

4.3.3.7 Storage layout after transform.

Processed keys follow `lichess-processed/{month}/` with separate folders for star-schema tables and the wide export. DuckDB holds a single file per project root (path from config) referencing those Parquet globs. Retrieval for training is therefore two-step: analysts query SQL; the ML CLI reads the exported wide Parquet and Feast saved datasets under `artifacts/lichess_features/`. I deliberately did not duplicate features in both DuckDB tables and Parquet more than necessary. Feast needs its registry, DuckDB needs SQL, sklearn needs a flat file.

4.3.3.8 Analytical processing angle.

The brief asks how data supports ML and investigation. SQL answers distribution questions (rating histograms, opening popularity by month). The Pandas pipeline encodes domain knowledge (Bullet vs Blitz, title hierarchies). Feast enforces temporal discipline. Together they mean preprocessing is not a single script but a chain where each tool does one job well. When GE fails, I inspect which expectation broke—often a new Lichess header field appeared—and patch transforms before re-running the DAG.

4.4 Model development

4.4.1 What

Model development trains a supervised classifier to predict game outcome from pre-game information such as ratings, titles, time control, opening family, and historical form. The business question that I was tackling was practical, if a player considers a given opening at their rating, what outcome distribution should they expect? Labels are three possible outcomes of white win, black win and draw. Deliverables include `model.joblib` on disk, metrics JSON, and a registered model in MLflow [27].

4.4.2 Why

Game-level rows are the natural ingestion grain, but players think in “my colour, my rating band, my opening”.  [28] expands each game to player-centric rows via `to_player_perspective` so the model answers that question.

4.4.3 How

Airflow task `"train_model"` calls `lichess-models train`. Inside  [29] where I wrap a scikit-learn [30] Pipeline  [31] which helps in building a `ColumnTransformer` for numeric and categorical columns, After which a classifier slot searched with `GridSearchCV` and `train test split`. Candidate estimators include logistic regression, random forest, and histogram gradient boosting; grids live in `PARAM_GRIDS`.

The best pipeline serialises to `artifacts/lichess_models/{run_id}/model.joblib`.  [32] scores the hold-out split and logs accuracy, macro-F1, and confusion-friendly counts to MLflow.  [33] publishes under the name `lichess-outcome-predictor`. The MLflow stack  [34] uses Postgres for metadata and MinIO (`mlflow-artifacts` bucket) for large artefacts, start `core` and `ml` profiles together or tracking breaks. I also kept notebooks under `notebook/` for exploratory work. The CLI path is what Airflow relies on.

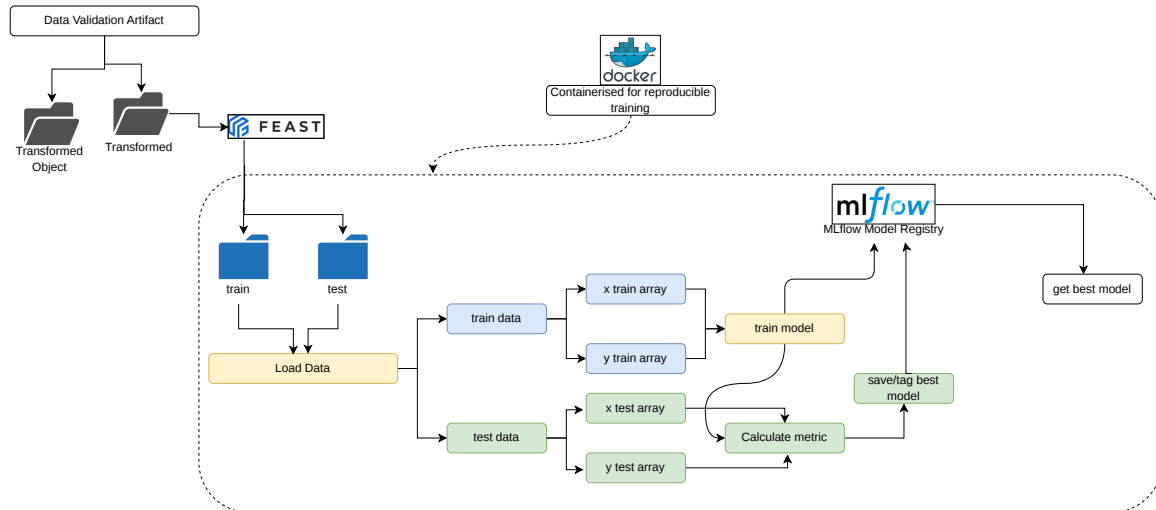


Figure 4.3: Model training: load splits, fit with hyperparameter search, log to MLflow.

The multi-class framing matches Lichess results better than treating draws as noise. I have not yet wired the opening weakness report ( `packages/lichess_models/src/lichess_models/analyze.py`) into the monthly DAG. It remains a manual analysis script for now.

4.5 Model deployment

4.5.1 What

Deployment here means exposing a trained model over HTTP so another service (or a human with curl) can request predictions without rerunning training. The FastAPI application [35] in  [36] serves `/health` and `/predict`. At startup it resolves a model URI `models:/lichess-outcome-predictor/Staging` from the MLflow registry.

4.5.2 Why

Serialising with `joblib` is simple. Registering the same object in MLflow ties the running API to an experiment lineage. When April's retrain beats March, It promotes a new version to staging.

4.5.3 How

I start the server with `uv run lichess-serving` from the repo root after exporting `MODEL_URI` and MLflow tracking variables. The CLI in  `packages/lichess_serving/src/lichess_serving/cli.py` wraps Uvicorn. Containerising is straightforward (slim Python image, same env vars) but left out of this submission to save integration time.

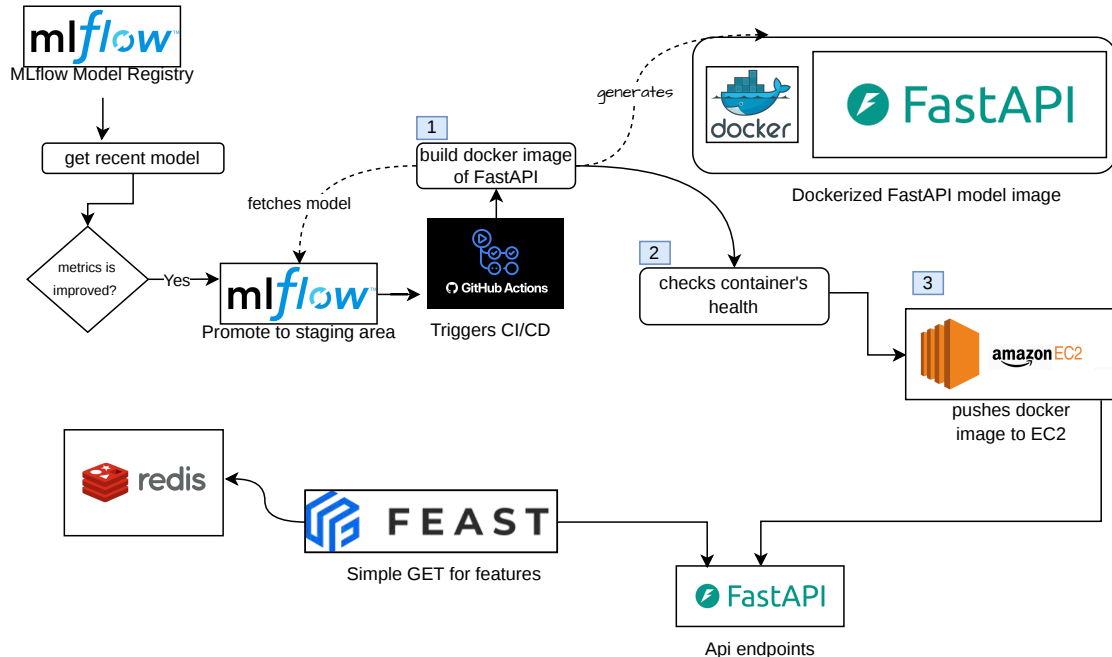


Figure 4.4: Deployment: promote model from MLflow, serve via FastAPI.

A minimal check I repeat after each promote: `GET /health` returns `loaded=true`, then `POST /predict` with a 1800-rated blitz game and the Sicilian Defence headers I used in tests. If probabilities sum to roughly one and the model gauge flips to 1, the deploy is good enough for demo purposes.

4.6 Model monitoring

4.6.1 What

Monitoring, in my setup, spans three layers: infrastructure metrics from the API, data-quality re-checks when a new month lands, and optional drift visualisation between reference and current feature files. Prometheus [37] scrapes the serving app; Grafana dashboards ship in the repo; Evidently [38] can render drift reports; Great Expectations reruns on fresh Parquet through `gcp[26]`.

4.6.2 Why

Accuracy on last month’s hold-out does not guarantee next month behaves. New openings surge after influencer games; rating inflation drifts feature means. Without `/metrics` I would not notice serving failures until a user complained. Without GE on the new shard I might train on a column that turned entirely null. Drift tooling is manual today, but it answers marker questions about “checking performance once deployed.”

4.6.3 How

`prometheus_fastapi_instrumentator` adds standard HTTP histograms. I implemented this `lichess_model_loaded` and `lichess_predictions_total` (counter) in [36] because they show up clearly on the dashboard JSON under `services/prometheus/grafana/provisioning/dashboards/json/serving.json`. Compose profile `monitoring` starts Prometheus on 9090 and Grafana on 3000; the scrape config [39] targets `host.docker.internal:8082` so a host-run server remains visible to containers.

After each monthly DAG run I would look at: (1) GE pass/fail in the artefact folder; (2) whether prediction rate spikes correlate with errors in `logs/lichess.log` via [40]; (3) optionally POST reference and current Parquet paths to the Evidently API [41] (evidently profile, port 5001) and open the HTML report in Streamlit on 8501. None of that is fully automated into Airflow yet.

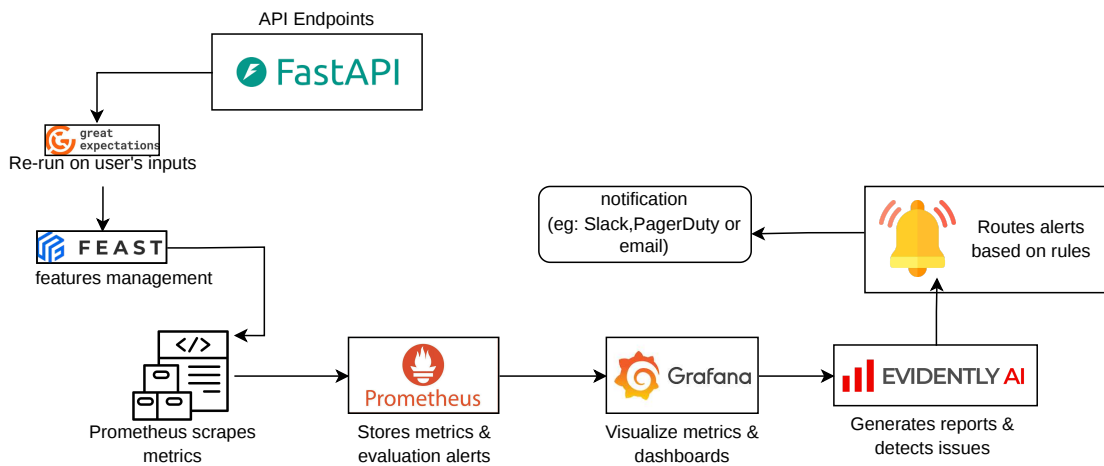


Figure 4.5: Monitoring: Prometheus, Grafana, drift reports, and pipeline alerts.

Alerts in a production system would route GE failures and Airflow task errors to email or Slack; here, a failed task email from Airflow’s built-in notifier is enough to document intent. The monitoring stage completes the loop back to ingestion: a new month triggers download, validation, and the same observability hooks repeat.

Chapter 5

Data analysis and insight generation / reporting

5.1 Introduction

Coursework Task 1 for CMP6230 was a planning exercise where I compared candidate datasets, chose the Lichess Open Database [1, 42], and sketched an MLOps pipeline with diagrams and storage justification. Task 2, documented here, is where that sketch became runnable software. This report is deliberately implementation-focused. I explain what I built, why I made particular storage and orchestration choices, and how each stage connects to code in the **LichessOps** GitHub repository [6].

Lichess publishes a full monthly dump of rated games as a single compressed archive named in (`lichess-YYYY-MM.pgn.zst`) template format. The file is large, the schema is chess-specific (PGN headers rather than relational tables), and the licence is permissive to do further downstream Machine learning tasks. That combination is learning opportunity rich for Data Management and Machine learning Operations(MLOPS) practice.

My pipeline treats each new month as an incremental batch of data. Two audiences motivated the design from Task 1. First is analysts who want SQL and notebooks over curated tables, and second are automated ML lifecycle paths that retrains when fresh data lands. I adopted an **ELT** pipeline rather than classic ETL. Due to ELT use, Extraction and load happen first in our pipeline. The raw `.pgn.zst` data lands in MinIO Object store exactly in the format that lichess uploads on `database.lichess.org` and transformation runs later in Spark and DuckDB is the columnar database where our finally computed and columnar parquet formats dataset is stored.

5.2 Evidence and reproducibility

The implementation lives in the public **LichessOps** repository [6]. Services are orchestrated with Docker Compose (MinIO, Airflow, Spark, DuckDB, MLflow, Prometheus, Grafana, and supporting databases). The monthly ingestion DAG can be triggered from the Airflow UI or by running the `lichess-data` CLI with the same arguments the DAG passes. Processed Parquet and DuckDB tables, Feast feature splits, MLflow runs, and the FastAPI serving endpoint provide traceable evidence for each pipeline stage.

Appendix B.1 indexes the main modules cited in Chapters 4 and 5. Markers can follow any claim in those chapters to a concrete file path on GitHub.

Chapter 6

Pipeline Evaluation and Monitoring

6.1 Introduction

Coursework Task 1 for CMP6230 was a planning exercise where I compared candidate datasets, chose the Lichess Open Database [1, 42], and sketched an MLOps pipeline with diagrams and storage justification. Task 2, documented here, is where that sketch became runnable software. This report is deliberately implementation-focused. I explain what I built, why I made particular storage and orchestration choices, and how each stage connects to code in the **LichessOps** GitHub repository [6].

Lichess publishes a full monthly dump of rated games as a single compressed archive named in (`lichess-YYYY-MM.pgn.zst`) template format. The file is large, the schema is chess-specific (PGN headers rather than relational tables), and the licence is permissive to do further downstream Machine learning tasks. That combination is learning opportunity rich for Data Management and Machine learning Operations(MLOPS) practice.

My pipeline treats each new month as an incremental batch of data. Two audiences motivated the design from Task 1. First is analysts who want SQL and notebooks over curated tables, and second are automated ML lifecycle paths that retrains when fresh data lands. I adopted an **ELT** pipeline rather than classic ETL. Due to ELT use, Extraction and load happen first in our pipeline. The raw `.pgn.zst` data lands in MinIO Object store exactly in the format that lichess uploads on `database.lichess.org` and transformation runs later in Spark and DuckDB is the columnar database where our finally computed and columnar parquet formats dataset is stored.

6.2 Evidence and reproducibility

The implementation lives in the public **LichessOps** repository [6]. Services are orchestrated with Docker Compose (MinIO, Airflow, Spark, DuckDB, MLflow, Prometheus, Grafana, and supporting databases). The monthly ingestion DAG can be triggered from the Airflow UI or by running the `lichess-data` CLI with the same arguments the DAG passes. Processed Parquet and DuckDB tables, Feast feature splits, MLflow runs, and the FastAPI serving endpoint provide traceable evidence for each pipeline stage.

Appendix B.1 indexes the main modules cited in Chapters 4 and 6. Markers can follow any claim in those chapters to a concrete file path on GitHub.

Chapter 7

Reflective Discussion

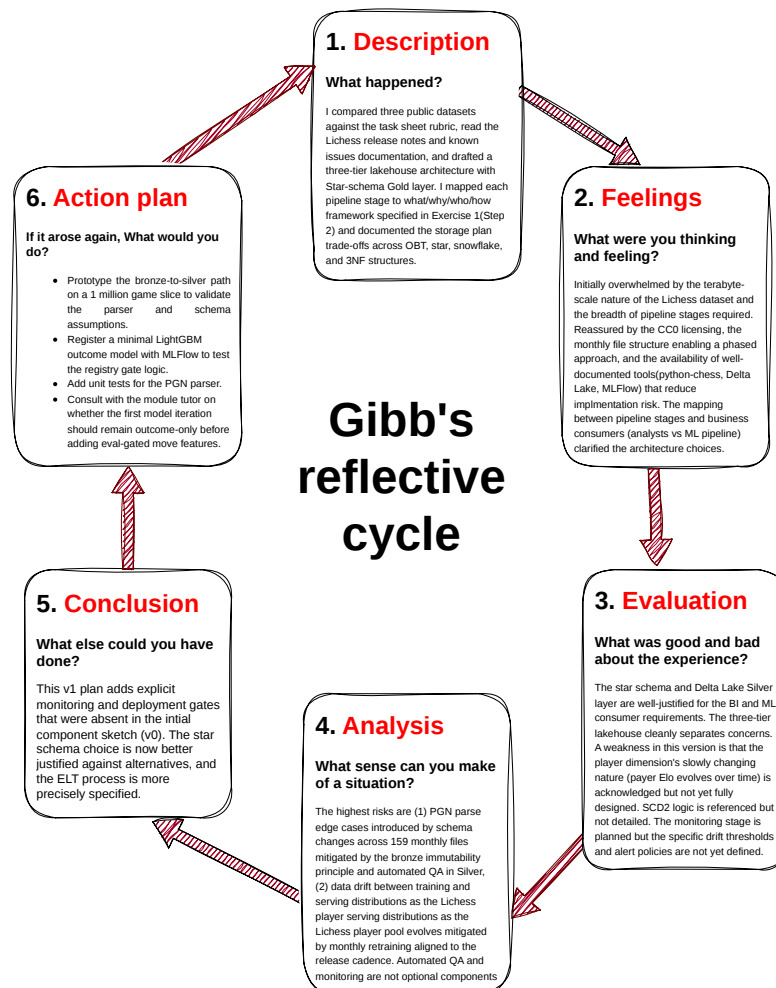


Figure 7.1: *Gibb's reflective cycle* applied to iterative planning and implementation for the Lichess MLOps pipeline.

Chapter 8

Conclusion, recommendation, and future work

8.1 Introduction

Coursework Task 1 for CMP6230 was a planning exercise where I compared candidate datasets, chose the Lichess Open Database [1, 42], and sketched an MLOps pipeline with diagrams and storage justification. Task 2, documented here, is where that sketch became runnable software. This report is deliberately implementation-focused. I explain what I built, why I made particular storage and orchestration choices, and how each stage connects to code in the **LichessOps** GitHub repository [6].

Lichess publishes a full monthly dump of rated games as a single compressed archive named in (`lichess-YYYY-MM.pgn.zst`) template format. The file is large, the schema is chess-specific (PGN headers rather than relational tables), and the licence is permissive to do further downstream Machine learning tasks. That combination is learning opportunity rich for Data Management and Machine learning Operations(MLOPS) practice.

My pipeline treats each new month as an incremental batch of data. Two audiences motivated the design from Task 1. First is analysts who want SQL and notebooks over curated tables, and second are automated ML lifecycle paths that retrains when fresh data lands. I adopted an **ELT** pipeline rather than classic ETL. Due to ELT use, Extraction and load happen first in our pipeline. The raw `.pgn.zst` data lands in MinIO Object store exactly in the format that lichess uploads on `database.lichess.org` and transformation runs later in Spark and DuckDB is the columnar database where our finally computed and columnar parquet formats dataset is stored.

8.2 Evidence and reproducibility

The implementation lives in the public **LichessOps** repository [6]. Services are orchestrated with Docker Compose (MinIO, Airflow, Spark, DuckDB, MLflow, Prometheus, Grafana, and supporting databases). The monthly ingestion DAG can be triggered from the Airflow UI or by running the `lichess-data` CLI with the same arguments the DAG passes. Processed Parquet and DuckDB tables, Feast feature splits, MLflow runs, and the FastAPI serving endpoint provide traceable evidence for each pipeline stage.

Appendix B.1 indexes the main modules cited in Chapters 4 and 8. Markers can follow any claim in those chapters to a concrete file path on GitHub.

Chapter 9

Further Plans for the Next Submission

In next submission, I will close the gaps identified. The most important are containerising the FastAPI server, automating Evidently drift checks in Airflow, and implementing retrain-on-drift. I will also add a canary metric gate to promote models through Staging to Production and wire opening weakness analysis into the API so the player-facing story in my draft is user-visible, not a script. Storage-wise I would document retention on `lichess-raw` keys (months are large) and test restore from backup.

Appendix A

Candidate dataset illustrations

```
Sample

[Event "Rated Bullet tournament https://lichess.org/tournament/yc1WW20x"]
[Site "https://lichess.org/PpwPOZMq"]
[Date "2017.04.01"]
[Round "-"]
[White "Abbot"]
[Black "Costello"]
[Result "0-1"]
[UTCDate "2017.04.01"]
[UTCTime "11:32:01"]
[WhiteElo "2100"]
[BlackElo "2000"]
[WhiteRatingDiff "-4"]
[BlackRatingDiff "+1"]
[WhiteTitle "FM"]
[ECO "B30"]
[Opening "Sicilian Defense: Old Sicilian"]
[TimeControl "300+0"]
[Termination "Time forfeit"]

1. e4 { [%eval 0.17] [%clk 0:00:30] } 1... c5 { [%eval 0.19] [%clk 0:00:30] }
2. Nf3 { [%eval 0.25] [%clk 0:00:29] } 2... Nc6 { [%eval 0.33] [%clk 0:00:30] }
3. Bc4 { [%eval -0.13] [%clk 0:00:28] } 3... e6 { [%eval -0.04] [%clk 0:00:30] }
4. c3 { [%eval -0.4] [%clk 0:00:27] } 4... b5? { [%eval 1.18] [%clk 0:00:30] }
5. Bb3?! { [%eval 0.21] [%clk 0:00:26] } 5... c4 { [%eval 0.32] [%clk 0:00:29] }
6. Bc2 { [%eval 0.2] [%clk 0:00:25] } 6... a5 { [%eval 0.6] [%clk 0:00:29] }
7. d4 { [%eval 0.29] [%clk 0:00:23] } 7... cxd3 { [%eval 0.6] [%clk 0:00:27] }
8. Qxd3 { [%eval 0.12] [%clk 0:00:22] } 8... Nf6 { [%eval 0.52] [%clk 0:00:26] }
9. e5 { [%eval 0.39] [%clk 0:00:21] } 9... Nd5 { [%eval 0.45] [%clk 0:00:25] }
10. Bg5?! { [%eval -0.44] [%clk 0:00:18] } 10... Qc7 { [%eval -0.12] [%clk 0:00:24] }
11. Nbd2?? { [%eval -3.15] [%clk 0:00:14] } 11... h6 { [%eval -2.99] [%clk 0:00:23] }
12. Bh4 { [%eval -3.0] [%clk 0:00:11] } 12... Ba6? { [%eval -0.12] [%clk 0:00:20] }
13. b3?? { [%eval -4.14] [%clk 0:00:02] } 13... Nf4? { [%eval -2.73] [%clk 0:00:19] }
```

Figure A.1: Excerpt from a Lichess-rated PGN archive: tournament headers, player ratings, opening metadata, SAN movelist, engine evaluations (%eval), and clock stamps (%clk).

Variable Name	Role	Type	Description	Units	Missing Values
fixed_acidity	Feature	Continuous			no
volatile_acidity	Feature	Continuous			no
citric_acid	Feature	Continuous			no
residual_sugar	Feature	Continuous			no
chlorides	Feature	Continuous			no
free_sulfur_dioxide	Feature	Continuous			no
total_sulfur_dioxide	Feature	Continuous			no
density	Feature	Continuous			no
pH	Feature	Continuous			no
sulphates	Feature	Continuous			no
alcohol	Feature	Continuous			no
quality	Target	Integer	score between 0 and 10		no
color	Other	Categorical	red or white		no

Figure A.2: UCI Wine Quality variable inventory: eleven continuous physicochemical features, ordinal quality target (0–10), and color stratum [3].

The Boston house-price data of Harrison, D. and Rubinfeld, D.L. 'Hedonic prices and the demand for clean air', J. Environ. Economics & Management, vol.5, 81-102, 1978. Used in Belsley, Kuh & Welsch, 'Regression diagnostics ...', Wiley, 1980. N.B. Various transformations are used in the table on pages 244-261 of the latter.

Variables in order:

CRIM per capita crime rate by town
 ZN proportion of residential land zoned for lots over 25,000 sq.ft.
 INDUS proportion of non-retail business acres per town
 CHAS Charles River dummy variable (= 1 if tract bounds river; 0 otherwise)
 NOX nitric oxides concentration (parts per 10 million)
 RM average number of rooms per dwelling
 AGE proportion of owner-occupied units built prior to 1940
 DIS weighted distances to five Boston employment centres
 RAD index of accessibility to radial highways
 TAX full-value property-tax rate per \$10,000
 PTRATIO pupil-teacher ratio by town
 B $1000(B_k - 0.63)^2$ where B_k is the proportion of blacks by town
 LSTAT % lower status of the population
 MEDV Median value of owner-occupied homes in \$1000's

0.00632	18.00	2.310	0	0.5380	6.5750	65.20	4.0900	1	296.0	15.30
396.90	4.98	24.00								
0.02731	0.00	7.070	0	0.4690	6.4210	78.90	4.9671	2	242.0	17.80
396.90	9.14	21.60								
0.02729	0.00	7.070	0	0.4690	7.1850	61.10	4.9671	2	242.0	17.80
392.83	4.03	34.70								
0.03237	0.00	2.180	0	0.4580	6.9980	45.80	6.0622	3	222.0	18.70
394.63	2.94	33.40								
0.06905	0.00	2.180	0	0.4580	7.1470	54.20	6.0622	3	222.0	18.70
396.90	5.33	36.20								
0.02985	0.00	2.180	0	0.4580	6.4300	58.70	6.0622	3	222.0	18.70
394.12	5.21	28.70								
0.08829	12.50	7.870	0	0.5240	6.0120	66.60	5.5605	5	311.0	15.20
395.60	12.43	22.90								
0.14455	12.50	7.870	0	0.5240	6.1720	96.10	5.9505	5	311.0	15.20
396.90	19.15	27.10								
0.21124	12.50	7.870	0	0.5240	5.6310	100.00	6.0821	5	311.0	15.20
386.63	29.93	16.50								
0.17004	12.50	7.870	0	0.5240	6.0040	85.90	6.5921	5	311.0	15.20
386.71	17.10	18.90								
0.22489	12.50	7.870	0	0.5240	6.3770	94.30	6.3467	5	311.0	15.20
392.52	20.45	15.00								
0.11747	12.50	7.870	0	0.5240	6.0090	82.90	6.2267	5	311.0	15.20
396.90	13.27	18.90								
0.09378	12.50	7.870	0	0.5240	5.8890	39.00	5.4509	5	311.0	15.20
390.50	15.71	21.70								
0.62976	0.00	8.140	0	0.5380	5.9490	61.80	4.7075	4	307.0	21.00
396.90	8.26	20.40								
0.63796	0.00	8.140	0	0.5380	6.0960	84.50	4.4619	4	307.0	21.00
380.02	10.26	18.20								
0.62739	0.00	8.140	0	0.5380	5.8340	56.50	4.4986	4	307.0	21.00
395.62	8.47	19.90								
1.05393	0.00	8.140	0	0.5380	5.9350	29.30	4.4986	4	307.0	21.00
386.85	6.58	23.10								
0.78420	0.00	8.140	0	0.5380	5.9900	81.70	4.2579	4	307.0	21.00
386.75	14.67	17.50								
0.80271	0.00	8.140	0	0.5380	5.4560	36.60	3.7965	4	307.0	21.00
288.99	11.69	20.20								
0.72580	0.00	8.140	0	0.5380	5.7270	69.50	3.7965	4	307.0	21.00
390.95	11.28	18.20								
1.25179	0.00	8.140	0	0.5380	5.5700	98.10	3.7979	4	307.0	21.00
376.57	21.02	13.60								

Figure A.3: Boston housing corpus header and fixed-width numeric rows: fourteen tract-level predictors with MEDV as the regression target [4, 5].

Appendix B

Lichess Open Database (reference material)

B.1 Code index

Table B.1 lists major implementation files with GitHub links (🔗). All paths are relative to repository root on branch main.

File	Description
🔗 <code>libs/shared/config_loader.py</code>	YAML config merge
🔗 <code>libs/shared/artifact_manager.py</code>	Artefact path resolution
🔗 <code>libs/shared/s3.py</code>	MinIO/S3 client helpers
🔗 <code>libs/shared/storage_config.py</code>	DuckDB and S3 URI mapping
🔗 <code>libs/shared/logger.py</code>	Rotating file logging
🔗 <code>services/airflow/dags/lichess_monthly_ingestion.py</code>	Monthly orchestration DAG
🔗 <code>packages/lichess_data/src/lichess_data/extract/lichess_downloader.py</code>	Shard download and checksum
🔗 <code>packages/lichess_data/src/lichess_data/validate/checks.py</code>	SHA256 validation
🔗 <code>packages/lichess_data/src/lichess_data/load/upload.py</code>	Raw upload to MinIO
🔗 <code>services/minio/docker-compose.yml</code>	Object store and buckets
🔗 <code>packages/lichess_data/src/lichess_data/spark/transform.py</code>	Spark star-schema transform
🔗 <code>packages/lichess_data/src/lichess_data/load/duckdb_sync.py</code>	Parquet → DuckDB sync
🔗 <code>packages/lichess_data/src/lichess_data/preprocessing/transforms.py</code>	Pandas feature stages
🔗 <code>packages/lichess_data/src/lichess_data/extract/pgn_parser.py</code>	Legacy PGN parser
🔗 <code>packages/lichess_features/src/lichess_features/split.py</code>	Feast split and temporal cut
🔗 <code>packages/lichess_data/src/lichess_data/validate/ge_runner.py</code>	Great Expectations runner
🔗 <code>packages/lichess_models/src/lichess_models/train.py</code>	Hyperparameter search training

File	Description
🔗 packages/lichess_models/src/lichess_models/features.py	Sklearn preprocessor
🔗 packages/lichess_models/src/lichess_models/dataset.py	Train/test loading
🔗 packages/lichess_models/src/lichess_models/evaluate.py	Hold-out metrics
🔗 packages/lichess_models/src/lichess_models/register.py	MLflow registration
🔗 services/mlflow/docker-compose.yml	MLflow tracking server
🔗 packages/lichess_serving/src/lichess_serving/app.py	FastAPI predict API
🔗 packages/lichess_serving/src/lichess_serving/schemas.py	API request/response models
🔗 services/prometheus/prometheus.yml	Metrics scrape config
🔗 services/evidently/src/app_fastapi/main.py	Drift report API

Table B.1: Implementation code index with GitHub links.

Bibliography

- [1] Lichess. *lichess.org open database*. 2026 URL: <https://database.lichess.org/>.
- [2] Paulo Cortez, António Cerdeira, Fernando Almeida, Telmo Matos and José Reis. ‘Modeling wine preferences by data mining from physicochemical properties’. In: *Decision Support Systems* 47.4 (2009). Smart Business Networks: Concepts and Empirical Evidence, pp. 547–553. ISSN: 0167-9236. DOI: <https://doi.org/10.1016/j.dss.2009.05.016> URL: <https://www.sciencedirect.com/science/article/pii/S0167923609001377>.
- [3] Paulo Cortez, António Cerdeira, Fernando Almeida, Telmo Matos and Joaquim Reis. *UCI Machine Learning Repository: Wine Quality Data Set*. <https://doi.org/10.24432/C56S3T>. 2009.
- [4] Daniel Harrison and Daniel L. Rubinfeld. ‘Hedonic housing prices and the demand for clean air’. In: *Journal of Environmental Economics and Management* 5.1 (1978), pp. 81–102. DOI: [10.1016/0095-0696\(78\)90006-2](https://doi.org/10.1016/0095-0696(78)90006-2).
- [5] StatLib / CMU. *Boston Housing Data Set*. <https://lib.stat.cmu.edu/datasets/boston>.
- [6] Nabin Oli. *LichessOps: Lichess MLOps pipeline implementation*. *GitHub repository — nabin2004/LichessOps*. 2026  GitHub: <https://github.com/nabin2004/LichessOps>.
- [7] Nabin Oli. *LichessOps: config_loader.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/libs/shared/config_loader.py. 2026.
- [8] Meta Platforms. *Zstandard compression*. 2026  GitHub: <https://github.com/facebook/zstd>.
- [9] MinIO, Inc. *MinIO — High Performance Object Storage*. 2026 URL: <https://min.io/>.
- [10] Nabin Oli. *LichessOps: MinIO docker-compose.yml*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/services/minio/docker-compose.yml>. 2026.
- [11] Apache Software Foundation. *Apache Airflow*. 2026 URL: <https://airflow.apache.org/>.
- [12] Nabin Oli. *LichessOps: lichess_monthly_ingestion.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/services/airflow/dags/lichess_monthly_ingestion.py. 2026.
- [13] Nabin Oli. *LichessOps: lichess_downloader.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/extract/lichess_downloader.py. 2026.
- [14] Nabin Oli. *LichessOps: upload.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/load/upload.py. 2026.

- [15] Nabin Oli. *LichessOps: s3.py*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/libs/shared/s3.py>. 2026.
- [16] Nabin Oli. *LichessOps: checks.py (checksum validation)*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/validate/checks.py. 2026.
- [17] Nabin Oli. *LichessOps: storage_config.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/libs/shared/storage_config.py. 2026.
- [18] Apache Software Foundation. *Apache Parquet*. 2026 URL: <https://parquet.apache.org/>.
- [19] Feast. *Feast — Feature Store for Machine Learning*. 2026 URL: <https://feast.dev/>.
- [20] Great Expectations. *Great Expectations*. 2026 URL: <https://greatexpectations.io/>.
- [21] Nabin Oli. *LichessOps: spark/transform.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/spark/transform.py. 2026.
- [22] Nabin Oli. *LichessOps: duckdb_sync.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/load/duckdb_sync.py. 2026.
- [23] Nabin Oli. *LichessOps: preprocessing/transforms.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/preprocessing/transforms.py. 2026.
- [24] Nabin Oli. *LichessOps: pgn_parser.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/extract/pgn_parser.py. 2026.
- [25] Nabin Oli. *LichessOps: lichess_features/split.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_features/src/lichess_features/split.py. 2026.
- [26] Nabin Oli. *LichessOps: ge_runner.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_data/src/lichess_data/validate/ge_runner.py. 2026.
- [27] LF AI & Data Foundation. *MLflow*. 2026 URL: <https://mlflow.org/>.
- [28] Nabin Oli. *LichessOps: lichess_models/dataset.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/dataset.py. 2026.
- [29] Nabin Oli. *LichessOps: lichess_models/train.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/train.py. 2026.
- [30] scikit-learn developers. *scikit-learn*. 2026 URL: <https://scikit-learn.org/>.
- [31] Nabin Oli. *LichessOps: lichess_models/features.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/features.py. 2026.

- [32] Nabin Oli. *LichessOps: lichess_models/evaluate.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/evaluate.py. 2026.
- [33] Nabin Oli. *LichessOps: lichess_models/register.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_models/src/lichess_models/register.py. 2026.
- [34] Nabin Oli. *LichessOps: MLflow docker-compose.yml*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/services/mlflow/docker-compose.yml>. 2026.
- [35] Sebastián Ramírez. *FastAPI*. 2026 URL: <https://fastapi.tiangolo.com/>.
- [36] Nabin Oli. *LichessOps: lichess_serving/app.py*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/packages/lichess_serving/src/lichess_serving/app.py. 2026.
- [37] Prometheus Authors. *Prometheus*. 2026 URL: <https://prometheus.io/>.
- [38] Evidently AI. *Evidently*. 2026 URL: <https://www.evidentlyai.com/>.
- [39] Nabin Oli. *LichessOps: prometheus.yml*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/services/prometheus/prometheus.yml>. 2026.
- [40] Nabin Oli. *LichessOps: logger.py*.  GitHub: <https://github.com/nabin2004/LichessOps/blob/main/libs/shared/logger.py>. 2026.
- [41] Nabin Oli. *LichessOps: Evidently drift API*.  GitHub: https://github.com/nabin2004/LichessOps/blob/main/services/evidently/src/app_fastapi/main.py. 2026.
- [42] Creative Commons. *CC0 1.0 Universal — Public Domain Dedication*. 2025 URL: <https://creativecommons.org/publicdomain/zero/1.0/>.